SALESTORM

High-Scale E-Commerce Scale

Stage 6 – SOLID & Design Patterns

1) Map SOLID principles to actual classes/components.

Principle	Actual Component/Class	How we use it
S – Single Responsibility	InventoryService	Handles only inventory/reservation logic
O – Open/Closed	PaymentStrategy	Add new payment providers without changing checkout
L – Liskov Substitution	PaymentProvider	Razorpay/Stripe/etc. can replace each other
I – Interface Segregation	PaymentProvider, NotificationService	Small focused interfaces
D – Dependency Inversion	CheckoutService	Depends on interfaces, not concrete providers

2) Use appropriate patterns such as Strategy, Factory, State, Observer, Adapter, Facade, Repository and Circuit Breaker.

Pattern	Used For	Problem Solved
Strategy	Payment / Pricing	Easily switch algorithms/providers
Factory	Payment Provider creation	Avoid if/else provider creation
State	Reservation/Order lifecycle	Controls valid state transitions
Observer	Notifications/events	Notify services without tight coupling
Adapter	External payment/delivery APIs	Integrates different APIs
Facade	Checkout	Provides one simple interface for complex workflow
Repository	Database access	Separates business logic from DB
Circuit Breaker	Payment/Order services	Prevents repeated calls to failed services

3) Explain the problem solved by each selected pattern and the trade-off introduced.

Pattern	Benefit	Trade-off
Strategy	Easy extension	More classes
Factory	Centralized creation	Extra abstraction
State	Safe lifecycle	More state classes
Observer	Loose coupling	Event debugging becomes harder
Adapter	Easy integration	Additional layer
Facade	Simple API	Can become too large
Repository	Clean DB separation	More code
Circuit Breaker	Failure protection	Requires monitoring/configuration

4) Show how the design can accommodate a new payment provider, pricing strategy or delivery partner with minimal change.





